- 📚 Prompt Library — hub_dtic_and_sis
  - Convenções
  - P01 — Análise e Diagnóstico Técnico
  - P02 — Planejamento e Roadmap
  - P03 — Extração de Conhecimento (pós-solução)
  - 📋 Registro de Conhecimento — Casos Documentados
    - [2026-03-09] — Hub / GLPI — Módulos Satélite Invisíveis após Migração de Perfis para Grupos
      - 1. Título
      - 2. Causa Raiz Confirmada
      - 3. Solução Aplicada
      - 4. Fundamento Técnico
      - 5. Como Detectar Mais Cedo
      - 6. Como Prevenir Recorrência
      - 7. Tags
    - [2026-03-09] — Hub / GLPI — Falhas Confirmadas e Pendentes de Validação
  - P02 — Roadmap: Consolidação da Matriz Permissional
    - Visão Geral
    - Fases
      - Fase A — Validação dos IDs e Criação dos Grupos no GLPI (< 1 hora)
      - Fase B — Teste Visual End-to-End da Sidebar e Rotas (30 min)
      - Fase C — Teste da Tela de Matriz Permissional (30 min)
      - Fase D — Correção de IDs (Condicional) (15 min)
      - Fase E — Documentação e Onboarding (1 hora)
    - Dependências entre Fases
    - Ordem de Execução Recomendada
    - Sequência de Ações Imediatas (Sem Código)

📚 Prompt Library — hub_dtic_and_sis

> Repositório central de prompts padronizados para uso no fluxo de desenvolvimento.
> Cada prompt é uma ferramenta reutilizável. Preencha os campos entre `[ ]` para cada uso.

Convenções

Campo	Significado
`CONTEXTO`	Cenário atual — o que existe, o que está em jogo
`OBJETIVO`	O que precisa ser alcançado nesta interação
`TAREFA`	Ações concretas que o agente deve executar
`FORMATO`	Como a resposta deve ser estruturada
`CRITÉRIOS`	O que define uma resposta válida e completa

P01 — Análise e Diagnóstico Técnico

> **Quando usar:** erro inesperado, comportamento anômalo, regressão, incidente em produção ou staging.

```
CONTEXTO
[Descreva o sistema afetado, o módulo ou funcionalidade envolvida, e o
comportamento observado.
 Inclua stack, linguagem, infra relevante se houver.]

SINTOMA
[O que o usuário ou sistema está reportando / exibindo.]

ERRO TÉCNICO
[Código HTTP, stack trace, log ou mensagem de erro exata, se disponível.]

OBJETIVO
Identificar a causa raiz do problema antes de qualquer alteração em código ou
configuração de produção.

TAREFA
1. Explicar tecnicamente o que esse erro/comportamento significa
2. Listar todos os pontos da arquitetura onde o problema pode estar originado
3. Mapear configurações ou dependências que tipicamente causam esse comportamento
4. Descrever um método de diagnóstico passo a passo para isolar a causa
5. Apresentar as estratégias de correção possíveis
6. Indicar riscos e trade-offs de cada abordagem

FORMATO DE SAÍDA
1. Diagnóstico técnico do problema
2. Hipóteses ordenadas por probabilidade
3. Pontos do sistema a verificar (com localização específica)
4. Roteiro de investigação
```

```
5. Opções de correção
6. Recomendação técnica fundamentada

CRITÉRIOS
- Nenhuma alteração deve ser sugerida sem diagnóstico confirmado
- Toda hipótese deve ter base técnica justificada
- A resposta deve permitir uma tomada de decisão informada
```

P02 — Planejamento e Roadmap

> **Quando usar:** nova funcionalidade, refatoração, migração, ou qualquer demanda que exige divisão em etapas antes de executar.

```
CONTEXTO
[Descreva o sistema ou módulo envolvido, o estado atual, e o que motivou essa
demanda.]

OBJETIVO
[Descreva claramente o estado final desejado — o que deve existir ou funcionar ao
final.]

RESTRIÇÕES
[Liste limitações conhecidas: tecnologia fixada, prazo, dependências externas,
risco de impacto em produção.]

TAREFA
1. Mapear o que já existe e o que precisa ser criado ou alterado
2. Identificar dependências entre etapas
3. Dividir o trabalho em fases executáveis e sequenciadas
4. Apontar riscos por fase
5. Indicar critérios de validação para cada etapa

FORMATO DE SAÍDA
1. Visão geral do plano
2. Fases com descrição, entregável e critério de conclusão
3. Dependências entre fases
4. Riscos mapeados
5. Ordem de execução recomendada

CRITÉRIOS
- O plano deve ser executável sem ambiguidade
- Cada fase deve ter um critério claro de "está feito"
- Nenhuma fase deve depender de suposições não confirmadas
```

P03 — Extração de Conhecimento (pós-solução)

> **Quando usar:** após qualquer solução validada — correção, implementação ou decisão arquitetural confirmada.
> Objetivo: transformar a solução em conhecimento persistido e reutilizável.

```
CONTEXTO DA SOLUÇÃO
[Descreva brevemente o problema que foi resolvido.]

SOLUÇÃO IMPLEMENTADA
[Descreva o que foi feito — mudança de código, configuração, decisão arquitetural.]

TAREFA
Com base no problema resolvido e na solução aplicada, gere um registro de
conhecimento com:

1. Título curto (formato: [SISTEMA] — [PROBLEMA RESOLVIDO])
2. Causa raiz confirmada
3. Solução aplicada (com trechos de código ou config se relevante)
4. Por que essa solução resolve o problema (fundamento técnico)
5. Como detectar esse problema mais cedo no futuro
6. Como prevenir recorrência
7. Tags de categorização (ex: #nginx #autorizacao #glpi #backend)

FORMATO DE SAÍDA
Bloco Markdown pronto para inserção no Knowledge Base do projeto.

CRITÉRIOS
- Linguagem técnica e direta
- Sem repetição de contexto já conhecido
- Deve ser compreensível por qualquer dev do time sem precisar do histórico da
conversa
```

📋 Registro de Conhecimento — Casos Documentados

> Cada entrada abaixo foi gerada ao final de uma solução validada.
> Formato: `[DATA] — [SISTEMA] — [PROBLEMA]`

[2026-03-09] — Hub / GLPI — Módulos Satélite Invisíveis após Migração de Perfis para Grupos

Status: ✅ Causa raiz confirmada | 🔧 Correção pendente de validação dos IDs

1. Título

Hub / GLPI — Módulos satélite invisíveis por ausência de grupos `Hub-App-*` no GLPI físico

2. Causa Raiz Confirmada

Os grupos `Hub-App-*` não existem na instância GLPI física. A função `resolve_app_access` (`auth_service.py:L22-43`) busca grupos com prefixo `Hub-App-` via API REST do GLPI e retorna `[]` quando nenhum é encontrado. Com `app_access = []`, o frontend (`resolveMenuItems` em `context-registry.ts:L116-141`) filtra e oculta todo módulo que tenha `requireApp` definido.

3. Solução Aplicada

Arquitetura implementada (dual-path):

Mecanismo	Resolve	Fonte	Arquivo
`resolve_hub_roles`	Papel funcional (`tecnico`, `gestor`, `solicitante`)	`profile_map` + `group_map` em `contexts.yaml`	`auth_service.py:L64-122`
`resolve_app_access`	Quais módulos satélite o usuário pode ver	Grupos `Hub-App-*` no GLPI via API	`auth_service.py:L22-43`

Código do extrator (`auth_service.py`):

```python
async def resolve_app_access(client, user_id: int) -> List[str]:
    group_links = await client.get_sub_items("User", user_id, "Group_User")
    app_access = []
    for gl in group_links:
        gid = gl.get("groups_id")
        group = await client.get_item("Group", gid)
        name = group.get("name", "")
        if name.startswith("Hub-App-"):
            app_access.append(name.replace("Hub-App-", "").lower())
    return app_access
```

Frontend bloqueador (`context-registry.ts`):

```typescript
// Em resolveMenuItems():
if (feature.requireApp && !appAccess.includes(feature.requireApp)) {
  return false; // Bloqueia imediatamente
}
```

4. Fundamento Técnico

Perfis e grupos no GLPI são entidades **independentes** com tabelas relacionais distintas (`glpi_profiles_users` vs `glpi_groups_users`). O Hub deliberadamente mantém dois caminhos paralelos: perfis definem o "quem você é" (role) e grupos `Hub-App-*` definem o "o que você pode acessar" (capability). Sem os grupos criados no GLPI, o segundo caminho retorna vazio e o frontend cumpre a trava de segurança projetada.

5. Como Detectar Mais Cedo

- Usar o endpoint de diagnóstico embutido: `GET /api/v1/{context}/auth/diagnose-access?username=X`
- Verificar campo `app_access` na resposta de `POST /auth/login` (DevTools > Network)
- Incluir log de nível `INFO` em `resolve_app_access` quando retornar lista vazia

6. Como Prevenir Recorrência

- Documentar no onboarding que grupos `Hub-App-*` são **pré-requisito** para visibilidade de módulos
- Criar checklist de deploy: "Grupos GLPI criados?" antes de liberar feature com `requireApp`

- Considerar fallback visual: módulo aparece bloqueado com tooltip "Solicite acesso ao administrador" em vez de sumir completamente

7. Tags

`#glpi` `#autorizacao` `#grupos` `#perfis` `#hub` `#frontend` `#backend` `#matriz-permissional`

[2026-03-09] — Hub / GLPI — Falhas Confirmadas e Pendentes de Validação

#	Falha	Status	Impacto
F1	Grupos `Hub-App-*` inexistentes no GLPI	✅ Confirmado	Todos os módulos com `requireApp` ficam invisíveis
F2	IDs dos grupos SIS (22, 21) podem divergir do `contexts.yaml`	⚠️ Não verificado	`tecnico-manutencao` e `tecnico-conservacao` não resolvem
F3	`glpigroups` pode retornar formato inesperado na sessão	⚠️ Não verificado	`groups[]` fica vazio silenciosamente
F4	DTIC não tem `group_map` no `contexts.yaml`	📋 Documentado	Sem impacto atual; bloqueia sub-roles futuras

---

P02 — Roadmap: Consolidação da Matriz Permissional

Visão Geral

Consolidar o sistema de controle de acesso do Hub baseado em dois mecanismos paralelos (Perfis → Roles, Grupos → Capabilities), garantindo que todos os módulos satélite obedeçam à matriz e que o administrador tenha uma interface visual para auditar e gerir permissões.

Fases

Fase A — Validação dos IDs e Criação dos Grupos no GLPI `(< 1 hora)`

Item	Detalhe
Descrição	Acessar o painel nativo do GLPI (SIS e DTIC) e (1) validar que os IDs dos grupos `CC-MANUTENCAO` e `CC-CONSERVACAO` coincidem com `22` e `21` do `contexts.yaml`; (2) criar os grupos `Hub-App-busca`, `Hub-App-carregadores`, `Hub-App-permissoes`; (3) atribuir ao usuário `jonathan-moletta`.
Entregável	Grupos existentes no GLPI + usuário atribuído
Critério de conclusão	`GET /auth/diagnose-access?username=jonathan-moletta` retorna `app_access: ["busca","carregadores","permissoes"]` e `hub_roles` contém `gestor`
Risco	Se IDs divergirem, `contexts.yaml` precisa ser atualizado antes de prosseguir
Dependência	Nenhuma

Fase B — Teste Visual End-to-End da Sidebar e Rotas `(30 min)`

Item	Detalhe
Descrição	Após Fase A, fazer login no Hub e verificar que: (1) Sidebar de Gestor DTIC mostra "Dashboard", "Smart Search", "Gestão de Acessos"; (2) Sidebar de Gestor SIS mostra "Dashboard", "Smart Search", "Gestão de Acessos", "Carregadores"; (3) Sidebar de Solicitante não mostra nenhum destes; (4) acesso direto por URL (`/dtic/permissoes`) sem a tag retorna Acesso Negado.
Entregável	Captura de tela confirmando visibilidade correta por role
Critério de conclusão	Todos os 4 cenários validados sem regressão
Risco	Baixo (leitura apenas)
Dependência	Fase A concluída

Fase C — Teste da Tela de Matriz Permissional `(30 min)`

Item	Detalhe
Descrição	Acessar `/dtic/permissoes` e `/sis/permissoes` e verificar: (1) tabela lista usuários do GLPI; (2) colunas de capability tags mostram ✅/❌ corretos; (3) coluna de roles resolvidas mostra papel correto; (4) botão "Editar Acesso" exibe instrução de redirecionamento GLPI.
Entregável	Tela funcional com dados reais
Critério de conclusão	Pelo menos 3 usuários visíveis com dados corretos de roles e app_access
Risco	Query SQL pode precisar ajuste se schema GLPI divergir
Dependência	Fases A + B concluídas

Fase D — Correção de IDs (Condicional) `(15 min)`

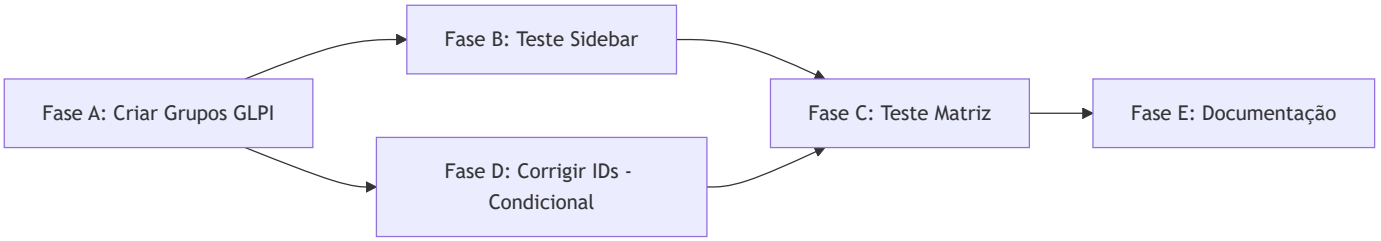
Item	Detalhe
Descrição	**Só executar se Fase A revelar IDs divergentes.** Atualizar `group_map` no `contexts.yaml` com os IDs reais. Rebuild do container backend.
Entregável	`contexts.yaml` com IDs corretos
Critério de conclusão	`resolve_hub_roles` retorna `tecnico-manutencao` para usuário no grupo correto
Risco	Baixo — alteração isolada em 2 números
Dependência	Fase A (diagnóstico dos IDs)

Fase E — Documentação e Onboarding `(1 hora)`

Item	Detalhe
Descrição	Atualizar documentação do projeto com: (1) checklist de grupos obrigatórios por contexto; (2) workflow de setup de novo usuário; (3) guia de troubleshooting usando `diagnose-access`.
Entregável	`docs/ONBOARDING_PERMISSOES.md`

Item	Detalhe
Critério de conclusão	Qualquer dev consegue configurar um novo usuário seguindo o guia
Risco	Nenhum
Dependência	Fases A-C concluídas

Dependências entre Fases



Ordem de Execução Recomendada

```
A → B → C → E
     ↘ D ↗ (somente se IDs divergirem)
```

Sequência de Ações Imediatas (Sem Código)

```
1. [LEITURA] Capturar payload real:
   GET /api/v1/dtic/auth/diagnose-access?username=jonathan-moletta

2. [LEITURA] Validar no painel GLPI (Administração > Grupos):
   - IDs reais dos grupos CC-MANUTENCAO e CC-CONSERVACAO
   - Confirmar se IDs batem com contexts.yaml (esperado: 22 e 21)

3. [AÇÃO] Criar grupos Hub-App-* no GLPI:
   DTIC: Hub-App-busca, Hub-App-permissoes
   SIS:  Hub-App-busca, Hub-App-carregadores, Hub-App-permissoes

4. [AÇÃO] Atribuir grupos ao usuário jonathan-moletta:
   DTIC: Hub-App-permissoes + Hub-App-busca
   SIS:  Hub-App-permissoes + Hub-App-busca + Hub-App-carregadores

5. [VALIDAÇÃO] Se IDs divergirem → atualizar contexts.yaml
```

Última atualização: 2026-03-09